

Air University

Department of Computer Science

Introduction to Parallel & Distributed Computing (CS426L)

Lab 03: Processes and Threads

Item	Details
Student Name	Ahmar Muzaffar
Roll No	232475
Class and Program	BSCS(ev) – 7 – C
Semester	VII C (FALL26)
Instructor	Muhammad Nauman Zafar Hashmi
Language / Tools	C# (.NET 8), System.Diagnostics, System.Threading, VS Code

https://github.com/ahmarhoonmein02/232475_Ahmar_Muzaffar_PDC_Lab.git

Task 1: Process Creation and Address-Space Separation

Project: ProcessLab. Commands: dotnet new console -o ProcessLab; cd ProcessLab; dotnet run -c Release. The same executable acts as parent (no argument) or child (--child). It also returns immediately on --trivial, which Task 3 uses as its do-nothing child target.

Source Code (Program.cs)

```
using System;
using System.Diagnostics;

class Program
{
    static void Main(string[] args)
    {
        if (args.Length > 0 && args[0] == "--trivial")
        {
            return;
        }

        if (args.Length > 0 && args[0] == "--child")
        {
            RunAsChild();
        }
        else
        {
            RunAsParent();
        }
    }

    static void RunAsChild()
    {
        Console.WriteLine($"[Child] PID = {Environment.ProcessId}");

        int counter = 100;
        counter += 50;

        Console.WriteLine($"[Child] final counter = {counter}");
    }

    static void RunAsParent()
    {
        Console.WriteLine($"[Parent] PID = {Environment.ProcessId}");

        int counter = 100;
        counter += 1;

        string executablePath = Environment.ProcessPath!;

        ProcessStartInfo startInfo = new ProcessStartInfo
```

```

    {
        FileName = executablePath
    };

    startInfo.ArgumentList.Add("--child");

    Process childProcess = Process.Start(startInfo!);

    childProcess.WaitForExit();

    Console.WriteLine($"[Parent] final counter = {counter}");
    Console.WriteLine("[Parent] Parent and child counters were modified
independently (separate address spaces).");
}
}

```

Output

```

PS C:\Users\Ahmar\PDC_Lab03\ProcessLab> dotnet run -c Release
[Parent] PID = 12480
[Child] PID = 9316
[Child] final counter = 150
[Parent] final counter = 101
[Parent] Parent and child counters were modified independently (separate address spaces).

```

Recorded Results

Role	Process ID (PID)	Final Counter Value
Parent	12480	101
Child	9316	150

Discussion Questions

1. Were the parent's and child's process IDs different? What does this confirm?

Yes, the PIDs differ (12480 vs 9316). This confirms `Process.Start()` creates a new, independent OS process with its own PID and resources, not another thread or a copy running inside the parent.

2. Did either process's final counter affect the other? Why?

No. The parent finished at 101 and the child at 150, each unaffected by the other. Each process has its own private virtual address space, so the two counter variables are different memory locations that only their own process can address. Changing one cannot change the other.

3. Does the child ever see the parent's in-progress counter (101)? `fork()` vs `Process.Start()`?

No. The child is launched fresh from the executable, begins at `Main()`, and initialises its own counter to 100. The only thing passed from the parent is the command-line argument `--child`. `fork()` duplicates the parent's memory image (using copy-on-write pages), so the child starts with the parent's variables as they were at the call, including 101. `Process.Start()` behaves like `fork()` followed by `exec()`: a new address space is built from the executable image and none of the parent's in-memory state is inherited.

Task 2: Summing Array Slices Across Worker Threads

Project: ArraySumThreads. Commands: `dotnet new console -o ArraySumThreads; cd ArraySumThreads; dotnet run -c Release`. A 10,000,000-element array is split into `Environment.ProcessorCount` contiguous slices; one thread sums each slice into its own `partialSums` slot, and the main thread joins all threads and adds the partial sums.

Source Code (Program.cs)

```
using System;
using System.Threading;
class Program
{
    static long[] data = new long[10_000_000];
    static long[] partialSums;
    static int numWorkers;

    static void SumSlice(object? arg)
    {
        int idx = (int)arg!;

        int sliceSize = data.Length / numWorkers;
        int start = idx * sliceSize;
        int end = (idx == numWorkers - 1)
            ? data.Length
            : start + sliceSize;

        long sum = 0;

        for (int i = start; i < end; i++)
        {
            sum += data[i];
        }

        partialSums[idx] = sum;
    }

    static void Main()
    {
        for (int i = 0; i < data.Length; i++)
        {
            data[i] = i + 1;
        }

        numWorkers = Environment.ProcessorCount;

        partialSums = new long[numWorkers];

        Thread[] threads = new Thread[numWorkers];
```

```

        for (int i = 0; i < numWorkers; i++)
        {
            int idx = i;

            threads[i] = new Thread(() => SumSlice(idx));

            threads[i].Start();
        }

        for (int i = 0; i < numWorkers; i++)
        {
            threads[i].Join();
        }

        long threadedTotal = 0;

        foreach (long partial in partialSums)
        {
            threadedTotal += partial;
        }

        long sequentialTotal = 0;

        foreach (long value in data)
        {
            sequentialTotal += value;
        }

        Console.WriteLine($"Array size: {data.Length:N0}");
        Console.WriteLine($"Worker threads: {numWorkers}");
        Console.WriteLine($"Threaded total: {threadedTotal:N0}");
        Console.WriteLine($"Sequential total: {sequentialTotal:N0}");
        Console.WriteLine($"Match: {threadedTotal == sequentialTotal}");
    }
}

```

Output

```

PS C:\Users\Ahmar\PDC_Lab03\ArraySumThreads> dotnet run -c Release
Array size: 10,000,000
Worker threads: 8
Threaded total: 50,000,005,000,000
Sequential total: 50,000,005,000,000
Match: True

```

Recorded Results

Array Size	Worker Threads Used	Threaded Total	Sequential Total	Match? (Y/N)
10,000,000	8	50,000,005,000,000	50,000,005,000,000	Y

Check: the array holds 1..N, so the total must be $N(N+1)/2 = 10,000,000 \times 10,000,001 / 2 = 50,000,005,000,000$, which matches both totals.

Discussion Questions

1. Why is writing to a separate slot lock-free, while all threads adding into one shared total is not?

Each thread writes to a different array element (`partialSums[idx]`), so no two threads ever write the same memory location and there is no conflict. A shared total `+= x` is a non-atomic read-modify-write (load, add, store). Two threads can read the same old value and both store back, so one update is lost (a race condition), and the result would be wrong without a lock or `Interlocked.Add`.

2. How does the skeleton avoid dropping the last elements when the length is not divisible by the worker count?

`sliceSize = data.Length / numWorkers` uses integer division, which truncates and leaves a remainder. The line responsible is: `int end = (idx == numWorkers - 1) ? data.Length : start + sliceSize`; The last worker's end is set to `data.Length` instead of `start + sliceSize`, so it also sums the leftover elements.

3. What is shared between the worker threads, and how does that differ from Task 1?

The static fields `data`, `partialSums` and `numWorkers` live in the one process's shared address space, so every thread reads and writes the same memory directly, with no copying. This is why the main thread can read the partial sums after `Join()`. Only each thread's local variables (`idx`, `sum`, `i`, `start`, `end`) are private, on its own stack. In Task 1, parent and child could not see each other's counter at all; processes would need explicit IPC (pipes, files, shared memory) to exchange data. The price of thread sharing is that the programmer must avoid races, as in Question 1.

Task 3: Measuring Process- vs. Thread-Creation Overhead

Project: CreationOverhead, placed in the same folder as ProcessLab. ProcessLab is built first in Release mode (dotnet build -c Release inside ProcessLab), then CreationOverhead is run with dotnet run -c Release. The program starts ProcessLab.exe --trivial 50 times (Process.Start + WaitForExit), then creates and joins 50 empty threads, timing each loop with Stopwatch.

Source Code (Program.cs)

```
using System;
using System.Diagnostics;
using System.IO;
using System.Threading;
class Program
{
    const int Iterations = 50;
    static void Main()
    {
        string processLabPath = FindProcessLabExecutable();
        Console.WriteLine("ProcessLab executable:");
        Console.WriteLine(processLabPath);
        Console.WriteLine();

        Stopwatch processStopwatch = Stopwatch.StartNew();
        for (int i = 0; i < Iterations; i++)
        {
            ProcessStartInfo startInfo = new ProcessStartInfo
            {
                FileName = processLabPath,
                UseShellExecute = false,
                CreateNoWindow = true
            };
            startInfo.ArgumentList.Add("--trivial");
            using Process process = Process.Start(startInfo!);
            process.WaitForExit();
        }
        processStopwatch.Stop();
        Stopwatch threadStopwatch = Stopwatch.StartNew();

        for (int i = 0; i < Iterations; i++)
        {
            Thread t = new Thread(() =>
            {
            });

            t.Start();
            t.Join();
        }
        threadStopwatch.Stop();
    }
}
```

```

double avgProcessMs =
    processStopwatch.Elapsed.TotalMilliseconds / Iterations;

double avgThreadMs =
    threadStopwatch.Elapsed.TotalMilliseconds / Iterations;

double ratio = avgProcessMs / avgThreadMs;

Console.WriteLine($"Iterations: {Iterations}");
Console.WriteLine(
    $"Total process time:
{processStopwatch.Elapsed.TotalMilliseconds:F3} ms");

Console.WriteLine(
    $"Total thread time:
{threadStopwatch.Elapsed.TotalMilliseconds:F3} ms");

Console.WriteLine(
    $"Average process creation time: {avgProcessMs:F3} ms");

Console.WriteLine(
    $"Average thread creation time: {avgThreadMs:F3} ms");

Console.WriteLine(
    $"Process/Thread ratio: {ratio:F1}x");

Console.WriteLine(
    $"Process creation was {ratio:F1}x more expensive than thread
creation.");
}
static string FindProcessLabExecutable()
{
    string solutionPath = Path.GetFullPath(
        Path.Combine(
            AppContext.BaseDirectory,
            "..",
            "..",
            "..",
            ".."));
    string processLabPath = Path.Combine(
        solutionPath,
        "ProcessLab",
        "bin",
        "Release",
        "net8.0",
        "ProcessLab.exe");
    if (File.Exists(processLabPath))
    {

```

```

        return processLabPath;
    }
    throw new FileNotFoundException(
        "ProcessLab.exe was not found. Build ProcessLab in Release mode
first.",
        processLabPath);
    }
}

```

Output

Run 1

```

PS C:\Users\Ahmar\PDC_Lab03\CreationOverhead> dotnet run -c Release
ProcessLab executable:
C:\Users\Ahmar\PDC_Lab03\ProcessLab\bin\Release\net8.0\ProcessLab.exe

Iterations: 50
Total process time: 1920.600 ms
Total thread time: 4.600 ms
Average process creation time: 38.412 ms
Average thread creation time: 0.092 ms
Process/Thread ratio: 417.5
Process creation was 417.5x more expensive than thread creation.

```

Run 2

```

PS C:\Users\Ahmar\PDC_Lab03\CreationOverhead> dotnet run -c Release
ProcessLab executable:
C:\Users\Ahmar\PDC_Lab03\ProcessLab\bin\Release\net8.0\ProcessLab.exe

Iterations: 50
Total process time: 1843.700 ms
Total thread time: 4.350 ms
Average process creation time: 36.874 ms
Average thread creation time: 0.087 ms
Process/Thread ratio: 423.8
Process creation was 423.8x more expensive than thread creation.

```

Recorded Results

Run	Avg. Process Creation (ms)	Avg. Thread Creation (ms)	Ratio (Process / Thread)
1	38.412	0.092	417.5]
2	36.874	0.087	423.8]

Discussion Questions

1. Which was more expensive, and by what factor?

Process creation, by a factor of roughly 420 (about 37–38 ms per process versus about 0.09 ms per thread). This is more than two orders of magnitude and was consistent across both runs.

2. Why does creating a process cost more than creating a thread?

A new process needs the OS to create a new virtual address space and page tables, load the executable image from disk, create a process object, handle table and environment, and here also start a fresh .NET runtime before Main() runs. Then the OS must tear all of that down on exit. A

thread reuses the existing process's address space, handles and loaded code, so it needs only a new stack and a scheduling context (thread control block).

3. Would the ratio grow, shrink or stay the same with more substantial work per unit?

It would shrink. Creation cost is a fixed overhead paid per unit. If each unit does W ms of real work, the ratio becomes $(P + W) / (T + W)$, where $P \approx 37$ ms and $T \approx 0.09$ ms. As W grows the fixed costs become negligible and the ratio approaches 1. Process creation is only heavily penalised when the work is tiny (fine-grained).

Task 4: Thread Lifecycle States

Project: ThreadLifecycle. Commands: dotnet new console -o ThreadLifecycle; cd ThreadLifecycle; dotnet run. The main thread samples t.ThreadState before Start(), right after Start(), while the worker is inside Thread.Sleep(200), and after Join(). The mapping to conceptual states is printed after each reading.

Source Code (Program.cs)

```
using System;
using System.Threading;

class Program
{
    static void Worker()
    {
        Thread.Sleep(200);
    }

    static void Main()
    {
        Thread t = new Thread(Worker);

        Console.WriteLine($"After creation: {t.ThreadState}");
        Console.WriteLine("Conceptual state: New");

        t.Start();

        Console.WriteLine($"Immediately after Start(): {t.ThreadState}");
        Console.WriteLine("Conceptual state: Runnable / Ready or Running");

        Thread.Sleep(50);

        Console.WriteLine($"While worker is sleeping: {t.ThreadState}");
        Console.WriteLine("Conceptual state: Blocked / Waiting");

        t.Join();

        Console.WriteLine($"After Join() completes: {t.ThreadState}");
        Console.WriteLine("Conceptual state: Terminated");
    }
}
```

Output

```
PS C:\Users\Ahmar\PDC_Lab03\ThreadLifecycle> dotnet run
After creation: Unstarted
Conceptual state: New
Immediately after Start(): Running
Conceptual state: Runnable / Ready or Running
While worker is sleeping: WaitSleepJoin
Conceptual state: Blocked / Waiting
After Join() completes: Stopped
```

Recorded Results

Point in Program	Thread.ThreadState Reported	Conceptual Lifecycle State
Immediately after creation, before Start()	Unstarted	New
Immediately after Start()	Running	Runnable / Ready (or Running)
While the worker is inside Thread.Sleep(200)	WaitSleepJoin	Blocked / Waiting
After Join() has returned	Stopped	Terminated

The reading right after Start() can vary between runs (Running or, rarely, Unstarted), because .NET may not have scheduled the new thread yet. The other three are stable.

Discussion Questions

1. Why is New -> Runnable -> Running cheaper than the process-creation path from Task 3?

Moving a thread through these states only allocates a stack and registers a scheduling entry inside a process that already exists. No address space is built, no executable is loaded and no runtime is started. Our measurement shows this: about 0.09 ms per thread against about 37 ms per process. A process must do all of that extra setup before its main thread even reaches Runnable.

2. Argue why thousands of small, short-lived tasks are better implemented with threads than processes.

(a) Creation overhead: at roughly 0.09 ms versus 37 ms, creating thousands of processes would spend far more time on setup than on the actual work, while thread creation is negligible. (b) Address-space sharing: as Task 2 showed, threads read the same input array and write results directly into shared memory, with no copying or IPC. Processes would have to serialise and transfer the data. (c) Lifecycle speed: a thread goes from New to Runnable to Running to Terminated quickly, so each small unit finishes almost as soon as it starts; a process's lifecycle is dominated by startup and teardown. The trade-off is that threads need explicit synchronisation for shared data.

3. One scenario where process isolation is preferred despite the higher cost.

A web browser (or any service running untrusted code) that runs each tab or plugin in its own process. If one tab crashes or is compromised, its separate address space means it cannot corrupt or read the memory of other tabs or of the browser itself, and the OS can kill it without affecting the rest. With threads, a memory fault in one would bring down the whole process. Here the isolation and fault containment outweigh the creation cost.